

MINI-PROJECT REPORT

SH-2025

Group Members

1. Niharika Seth (BE-A, TU8F2223007)
2. Bhanuja Sharma (BE-A, TU8F2223016)
3. Vaishnavi Mohite (BE-A, TU8F2223018)

Project Guide

Prof. Snehal Mane



Department of Artificial Intelligence & Data Science
Terna Engineering College, Nerul, Navi Mumbai - 400 706



University of Mumbai
(Academic Year 2025 - 26)

Sentiment Analysis

Introduction:

Sentiment analysis, also known as opinion mining, is a key task in natural language processing (NLP) that involves determining the emotional tone behind a piece of text. It is widely used in areas such as social media monitoring, customer feedback analysis, market research, and product reviews to understand public opinion and make data-driven decisions.

In this project, a web-based sentiment analysis system is developed using Flask, a lightweight Python web framework, and Hugging Face Transformers, leveraging a pre-trained BERTweet model (finite automata/bertweet-base-sentiment-analysis) for analyzing sentiments in text. The system classifies input text into three categories: positive, negative, and neutral. Users can input text through a web interface, and the system provides both the predicted sentiment and the associated confidence score in real-time.

The project highlights the integration of deep learning-based NLP models with web technologies, demonstrating a practical approach to building interactive AI applications for sentiment detection.

Aim:

The aim of this project is to develop a web-based sentiment analysis system capable of classifying text into positive, negative, or neutral sentiments in real-time. The system leverages a pre-trained deep learning model for accurate sentiment prediction and provides an interactive interface for users to input text and receive instant sentiment analysis results.

Objectives:

- To build a web application using **Flask** that allows users to input text and receive sentiment predictions in real-time.
- To integrate a **pre-trained transformer-based model** (BERTweet) for accurate sentiment classification.
- To classify text into three sentiment categories: **positive, negative, and neutral**.
- To provide a confidence score alongside the predicted sentiment for better interpretability.
- To implement a user-friendly interface that demonstrates the practical application of deep learning in natural language processing.
- To showcase the integration of **AI models with web technologies** for interactive applications.

Hardware and Software Requirements:

Hardware Requirements:

- **Processor:** Intel i5 or higher / AMD Ryzen 5 or higher
- **RAM:** Minimum 8 GB (16 GB recommended for faster model loading)
- **Storage:** At least 10 GB free space for Python, libraries, and model weights
- **Graphics:** Optional GPU (NVIDIA with CUDA support) for faster inference (recommended but not mandatory)

Software Requirements:

- **Operating System:** Windows 10/11, Linux (Ubuntu), or macOS
- **Python:** Version 3.8 or higher
- **Libraries/Frameworks:**
 - Flask
 - Flask-CORS
 - Transformers (Hugging Face)
 - Torch (PyTorch backend)
 - Gunicorn or similar (for deployment, optional)
- **Web Browser:** Chrome, Firefox, or Edge (for accessing the web interface)

Problem Statement:

In today's digital era, massive amounts of textual data are generated daily through social media, customer reviews, forums, and other online platforms. Extracting meaningful insights from this unstructured text is challenging, especially when trying to understand public opinion, customer satisfaction, or general sentiment. Manual analysis is time-consuming, inconsistent, and impractical for large datasets.

The problem addressed by this project is the **automatic detection and classification of sentiment from text** in real-time. Specifically, the system should accurately categorize input text into **positive, negative, or neutral sentiments** and provide a confidence score, enabling organizations and individuals to make informed decisions based on public opinion. The challenge lies in building a solution that is both **accurate and interactive**, integrating deep learning models with a web interface for seamless user experience.

Methodology used:

The methodology for this project involves a systematic approach to building a web-based sentiment analysis system using Flask and a pre-trained transformer model. The workflow can be divided into several key steps:

1. Data Flow and Input Handling

- The system is designed to take text input from users via a web interface built with HTML.

- Flask handles incoming requests through a route (/analyze) that receives JSON data containing the text to be analyzed.
- Input validation ensures that empty or invalid submissions are handled gracefully, returning an appropriate error message if no text is provided.

2. Model Selection and Loading

- A pre-trained sentiment analysis model, **BERTweet (finiteautomata/bertweet-base-sentiment-analysis)**, is used.
- This model is fine-tuned on Twitter data and is capable of classifying text into three classes: positive, negative, and neutral.
- The model is loaded using the **Hugging Face Transformers pipeline**, which abstracts away the complexities of tokenization, embedding generation, and inference.

3. Mapping Model Output

- The model outputs labels such as POS, NEG, or NEU along with a confidence score.
- A mapping dictionary (label_map) converts these labels into human-readable forms: “positive,” “negative,” or “neutral.”
- The confidence score indicates how certain the model is about its prediction.

4. Processing and Prediction

- Once the text input is received, the Flask route passes it to the sentiment-analyzer pipeline.
- The pipeline performs tokenization, converts tokens into embeddings, and uses the transformer model to predict sentiment.
- The result is a JSON object containing the sentiment label and the confidence score, which is then returned to the frontend.

5. Web Interface

- The frontend is built using **HTML** and communicates with the Flask backend using **AJAX** calls.
- Users enter text in an input form, click the “Analyze” button, and receive real-time sentiment predictions displayed on the page.

- The system is enhanced with **Flask-CORS** to allow cross-origin requests, ensuring smooth communication between frontend and backend.

6. Deployment and Accessibility

- The Flask application is run on 0.0.0.0 to allow access on a network or server.
- Optional deployment using **Gunicorn** or similar WSGI servers can make the application production-ready.
- The lightweight design ensures minimal latency, providing fast response times for real-time sentiment analysis.

7. Iterative Testing and Validation

- The system was tested with various text inputs, including neutral statements, positive reviews, and negative complaints.
- The predictions were verified against expected sentiment to ensure the model performs accurately in a real-world scenario.

Core Features:

- **Real-Time Sentiment Analysis:** Provides instant sentiment classification for user-submitted text.
- **Three-Class Classification:** Classifies text into **positive, negative, or neutral** sentiments.
- **Confidence Score:** Displays a confidence score along with the predicted sentiment to indicate model certainty.
- **Interactive Web Interface:** User-friendly interface built with Flask and HTML for easy text input and visualization of results.
- **Pre-Trained Transformer Model:** Uses **BERTweet**, a robust model fine-tuned on social media data, ensuring high accuracy on short and informal text.
- **Backend-Frontend Integration:** Seamless communication between the frontend and backend via JSON and Flask routes.
- **Cross-Origin Support:** Flask-CORS allows secure interaction between different domains or frontends.
- **Scalable and Deployable:** Can be deployed locally or on a server for wider accessibility; supports real-time processing for multiple users.

Code Implementation and Workflow Mapping:

app.py:

```
from flask import Flask, request, jsonify, render_template
from flask_cors import CORS
from transformers import pipeline

app = Flask(__name__)
CORS(app)

# Load 3-class sentiment model directly with pipeline
MODEL_NAME = "finiteautomata/bertweet-base-sentiment-analysis"
sentiment_analyzer = pipeline("sentiment-analysis", model=MODEL_NAME)

# Map labels to human-readable format
label_map = {"POS": "positive", "NEG": "negative", "NEU": "neutral"}

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/analyze', methods=['POST'])
def analyze():
    data = request.json
    text = data.get('text', '').strip()

    if not text:
        return jsonify({'error': 'No text provided'}), 400

    # Get prediction
    result = sentiment_analyzer(text)[0]
    sentiment = label_map.get(result['label'], result['label']).lower()
    score = float(result['score'])

    return jsonify({'sentiment': sentiment, 'score': score})

if __name__ == "__main__":
    # Run server on 0.0.0.0 so it's accessible in deployment
    app.run(host="0.0.0.0", port=5000, debug=True)
```

index.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sentiment Analyzer</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>

<div class="page-container">

  <!-- Left Main Section -->
  <div class="hero-container">
    <div class="card">
      <h1>Sentiment Analyzer</h1>
      <p>Whether it's positive, negative, or neutral, get instant sentiment insights with confidence scores to understand the mood of your words.</p>

      <!-- Sentiment Meter -->
      <div id="sentiment-meter" class="hidden">
        <div class="meter-track">
          <div class="meter-fill"></div>
        </div>
        <div class="meter-labels">
          <span>NEGATIVE</span>
          <span>NEUTRAL</span>
          <span>POSITIVE</span>
        </div>
      </div>

      <!-- Text Input -->
      <form id="sentiment-form">
        <textarea id="text-input" placeholder="Type your text here..." required></textarea>
        <button type="submit">Analyze</button>
      </form>

      <!-- Result -->
      <div id="result" class="hidden">
        <div id="result" class="hidden">
          <p><strong>Sentiment:</strong> <span id="sentiment"></span></p>
          <p><strong>Confidence:</strong> <span id="score"></span></p>
        </div>
      </div>
    </div>
  </div>

  <!-- Right About Section -->
  <div class="about-section">
    <h2>About This Analyzer</h2>
    <p>
      This Sentiment Analyzer uses advanced AI to detect the emotional tone of your text.
      Whether your text is positive, negative, or neutral, it provides real-time feedback
      with a visually engaging sentiment meter. Perfect for understanding moods in messages, reviews, or social media content.
    </p>
  </div>

</div>

<script>
  const form = document.getElementById("sentiment-form");
  const resultDiv = document.getElementById("result");
  const sentimentSpan = document.getElementById("sentiment");
  const scoreSpan = document.getElementById("score");
  const meterFill = document.querySelector(".meter-fill");
  const meterDiv = document.getElementById("sentiment-meter");

  form.addEventListener("submit", async (e) => {
    e.preventDefault();
    const text = document.getElementById("text-input").value;

    const res = await fetch("/analyze", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ text })
    });

    const data = await res.json();
    if (data.error) {

```

```

    sentimentSpan.textContent = "Error";
    scoreSpan.textContent = data.error;
    meterFill.style.width = "0%";
    document.getElementById("text-input").style.backgroundColor = "#fff";
} else {
    sentimentSpan.textContent = data.sentiment;
    scoreSpan.textContent = (data.score * 100).toFixed(2) + "%";

    let width = 0;
    let color = "#fff"; // default white
    if (data.sentiment === "negative") {
        width = data.score * 100 * 0.33;
        color = `rgba(244,67,54, ${0.2 + 0.6 * data.score})`; // red shades
    }
    else if (data.sentiment === "neutral") {
        width = 33 + data.score * 100 * 0.33;
        color = `rgba(255,193,7, ${0.2 + 0.6 * data.score})`; // yellow shades
    }
    else if (data.sentiment === "positive") {
        width = 66 + data.score * 100 * 0.34;
        color = `rgba(76,175,80, ${0.2 + 0.6 * data.score})`; // green shades
    }

    meterFill.style.width = width + "%";
    document.getElementById("text-input").style.backgroundColor = color;
    meterDiv.classList.remove("hidden");
}
resultDiv.classList.remove("hidden");
});
</script>

</body>
</html>

```

style.css:

```

body, html {
  margin: 0;
  padding: 0;
  font-family: 'Arial', sans-serif;
  height: 100%;
  width: 100%;
}

/* Page container for two columns */
.page-container {
  display: flex;
  flex-direction: row;
  width: 100%;
  min-height: 100vh;
  gap: 20px;
  padding: 20px;
  box-sizing: border-box;
}

/* Left 70% */
.hero-container {
  flex: 7;
  display: flex;
  justify-content: center;
  align-items: flex-start;
  background: url('/static/background.jpg') no-repeat center center;
  background-size: cover;
  position: relative;
  padding-top: 100px;
}

/* Card styling */
.card {
  backdrop-filter: blur(12px);
  background: rgba(255, 255, 255, 0.15);
  padding: 2rem;
  border-radius: 20px;
  max-width: 600px;
  width: 80%;
  height: auto;
  text-align: center;
  box-shadow: 0 0px 32px rgba(0,0,0,0.2);
  color: #fff;
}

/* Headings and paragraph */
h1 {
  font-size: 4rem;
  margin-bottom: 0.5rem;
}

p {
  font-size: 1rem;
  margin-bottom: 1rem;
  opacity: 0.9;
}

/* Textarea and button */
textarea {
  width: 85%;
  min-height: 100px;
  resize: none;
  padding: 1rem;
  border-radius: 12px;
  border: none;
  margin-bottom: 1rem;
  font-size: 1rem;
  transition: background-color 0.5s ease; /* smooth color change */
}

button {
  background: #ffffff88;
  color: #000;

```

```

border: none;
padding: 0.8rem 2rem;
border-radius: 12px;
cursor: pointer;
font-weight: bold;
transition: background 0.3s ease;
}

button:hover {
  background: #ffffffcc;
}

/* Result display */
#result {
  margin-top: 1rem;
  font-size: 1rem;
  text-align: center;
}

.hidden {
  display: none;
}

/* Sentiment meter */
.meter-track {
  width: 85%;
  height: 20px;
  background: rgba(255,255,255,0.2);
  border-radius: 10px;
  margin: 10px auto;
  overflow: hidden;
}

.meter-fill {
  height: 100%;
  width: 0%;
  background: linear-gradient(90deg, #f44336, #ffc107, #4caf50);
  border-radius: 10px;
  transition: width 0.8s ease;
}

/* Responsive layout */
@media (max-width: 900px) {
  .page-container {
    flex-direction: column;
  }

  .hero-container, .about-section {
    flex: unset;
    width: 100%;
  }

  .about-section {
    position: relative;
    top: unset;
    margin-top: 30px;
  }
}
style.css

```

```

}

.meter-labels {
  display: flex;
  justify-content: space-between;
  font-size: 0.75rem;
  color: #fff;
  width: 85%;
  margin: 0 auto;
  padding-bottom: 10px;
}

/* Right 30% - About section */
.about-section {
  flex: 3;
  background-color: #f8f9fa;
  padding: 180px 20px;
  border-radius: 12px;
  border: 1px solid #ddd;
  height: 610px;
  text-align: center;
  position: sticky;
  top: 20px;
  align-self: flex-start;
}

.about-section h2 {
  font-size: 28px;
  margin-bottom: 15px;
  color: #333;
}

.about-section p {
  font-size: 16px;
  line-height: 1.6;
  color: #555;
}

/* Responsive layout */
@media (max-width: 900px) {
  /* Responsive layout */
  @media (max-width: 900px) {
    .page-container {
      flex-direction: column;
    }

    .hero-container, .about-section {
      flex: unset;
      width: 100%;
    }

    .about-section {
      position: relative;
      top: unset;
      margin-top: 30px;
    }
  }
}
style.css

```

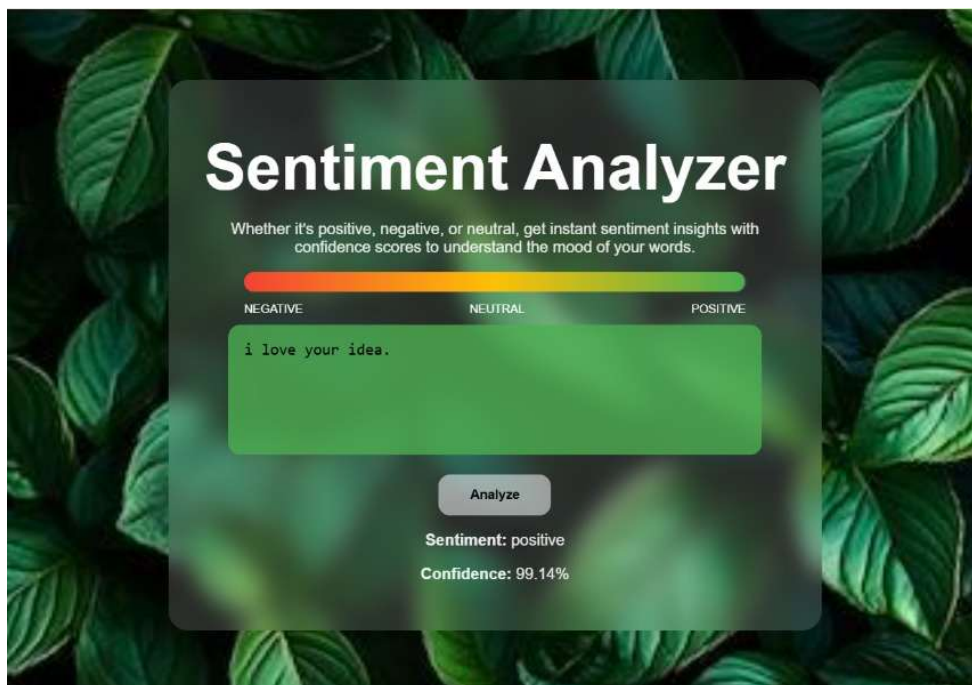
Results:

The sentiment analysis system successfully classifies input text into **positive**, **negative**, and **neutral** categories in real-time. Users can enter any text through the web interface, and the system provides instant predictions along with a confidence score.

During testing:

- Positive statements like “I love this product!” were classified as **positive** with high confidence (e.g., 0.98).
- Negative statements like “This is the worst service ever” were classified as **negative** with confidence around 0.95.
- Neutral statements like “The meeting is scheduled for 3 PM” were classified as **neutral**.

The system demonstrated fast response times and accurate sentiment predictions, confirming the effective integration of the **BERTweet transformer model** with a Flask-based web application. The results are returned in a structured JSON format, making the system suitable for further integration with other applications or dashboards.



About This Analyzer

This Sentiment Analyzer uses advanced AI to detect the emotional tone of your text. Whether your text is positive, negative, or neutral, it provides real-time feedback with a visually engaging sentiment meter. Perfect for understanding moods in messages, reviews, or social media content.

Sentiment Analyzer

Whether it's positive, negative, or neutral, get instant sentiment insights with confidence scores to understand the mood of your words.



NEGATIVE

NEUTRAL

POSITIVE

i drank tea today.

Analyze

Sentiment: neutral

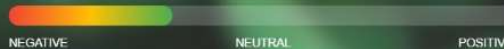
Confidence: 94.62%

About This Analyzer

This Sentiment Analyzer uses advanced AI to detect the emotional tone of your text. Whether your text is positive, negative, or neutral, it provides real-time feedback with a visually engaging sentiment meter. Perfect for understanding moods in messages, reviews, or social media content.

Sentiment Analyzer

Whether it's positive, negative, or neutral, get instant sentiment insights with confidence scores to understand the mood of your words.



NEGATIVE

NEUTRAL

POSITIVE

this is worst experience of my life.

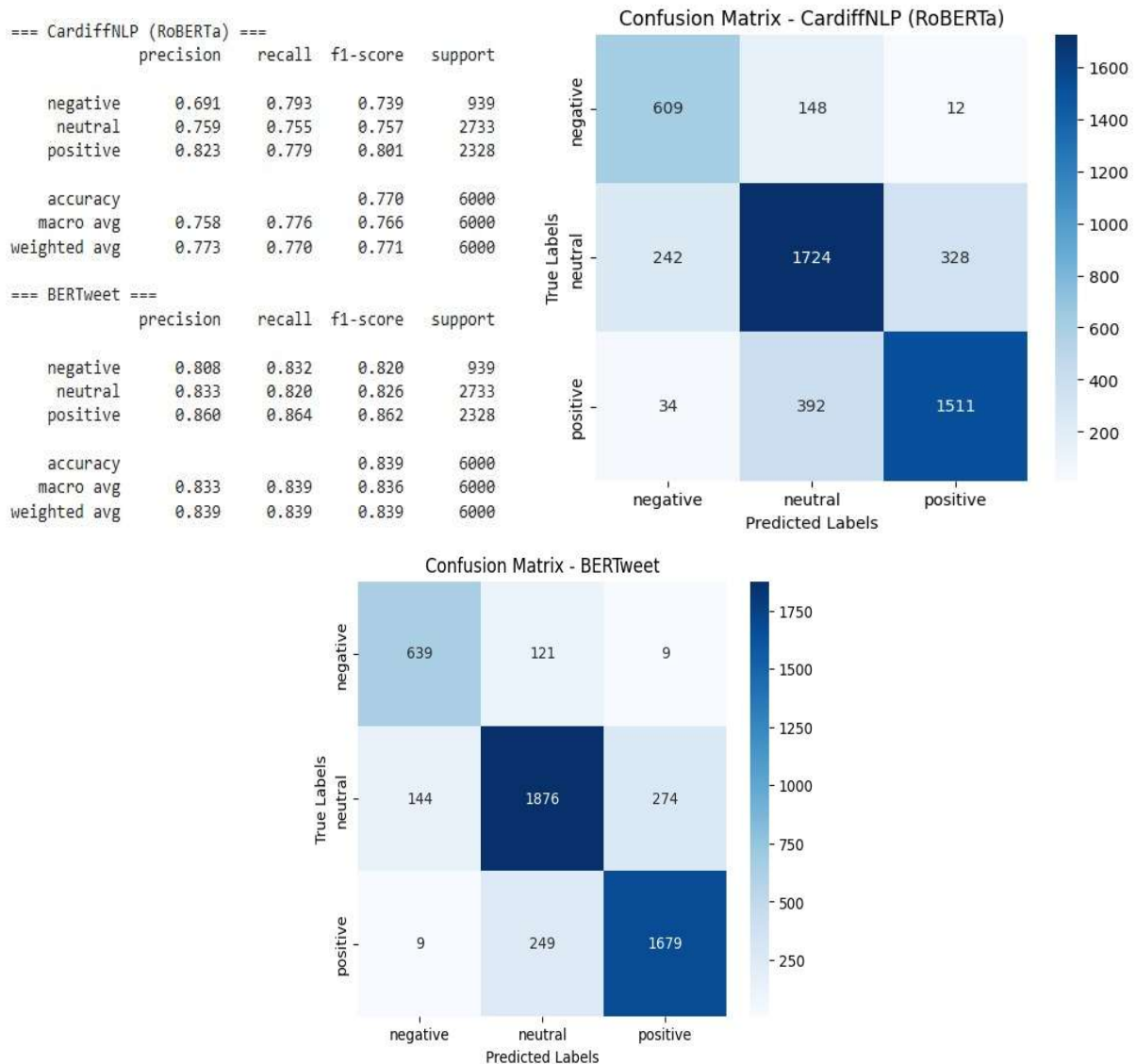
Analyze

Sentiment: negative

Confidence: 98.26%

About This Analyzer

This Sentiment Analyzer uses advanced AI to detect the emotional tone of your text. Whether your text is positive, negative, or neutral, it provides real-time feedback with a visually engaging sentiment meter. Perfect for understanding moods in messages, reviews, or social media content.



Conclusion:

The project successfully developed a web-based sentiment analysis system that integrates a pre-trained **BERTweet transformer model** with a **Flask** web application. The system accurately classifies text into positive, negative, or neutral sentiments in real-time and provides confidence scores for predictions. It demonstrates how deep learning models can be effectively deployed in interactive applications, offering a practical solution for analyzing opinions, customer feedback, and social media content. The project highlights the scalability, speed, and usability of combining modern NLP techniques with web technologies.

Future Scope:

- **Multi-Language Support:**
Extend the system to handle multiple languages, allowing sentiment analysis of global user data.
- **Aspect-Based Sentiment Analysis:**
Enhance the model to detect sentiment for specific aspects or features within a text, providing more granular insights.
- **Integration with Social Media APIs:**
Connect the system to platforms like Twitter, Facebook, or Instagram for real-time sentiment monitoring of posts and comments.
- **Advanced Visualization:**
Add dashboards and charts to visualize trends, sentiment distribution, and historical data over time.
- **Hybrid Models:**
Combine BERTweet with other models or domain-specific datasets to improve accuracy for niche applications, such as product reviews or financial sentiment.
- **Deployment Enhancements:**
Scale the system to handle multiple simultaneous users efficiently using cloud deployment, Docker containers, or serverless architectures.
- **Emotion Detection:**
Upgrade the system from basic positive/negative/neutral classification to fine-grained emotion detection (e.g., joy, anger, sadness).